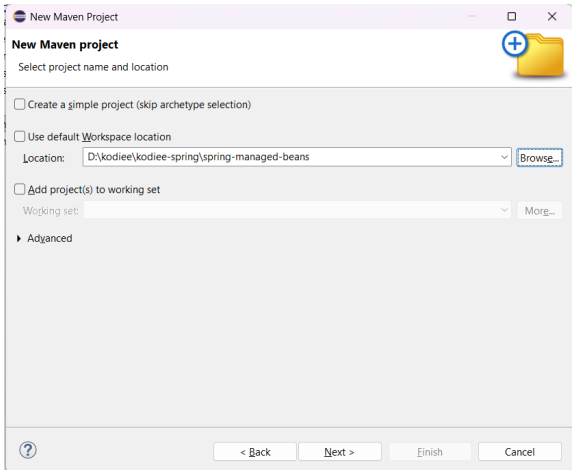
Employee Management Application Using Java-Based Configuration

Project: employee-management-application-using-java-type

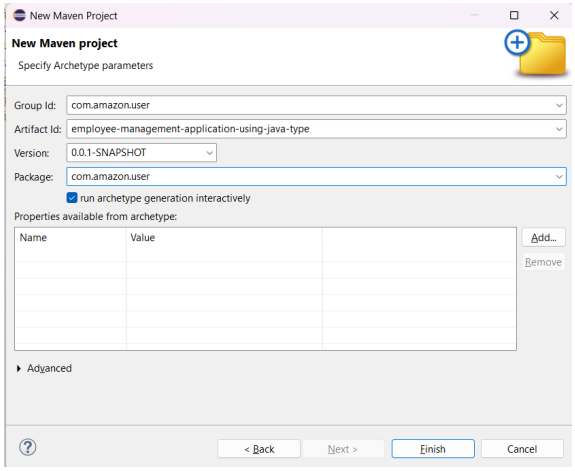
Step 1: Create the Maven project in Eclipse

- File → New → Maven Project → set the workspace location path → Next



Workspace location

- Set Catalog to Internal and select the archetype → Next
- Set Group Id (`com.amazon.user`), Artifact Id (`employee-management-application-using-java-type`), Version (`0.0.1-SNAPSHOT`) and Package (`com.amazon.user`) → Finish



Archetype parameters

- Type Y in the Console when prompted

Step 2: Add the Spring Context dependency

A project is created in Eclipse's Package Explorer with the name of the artifact we set, `employee-management-application-using-java-type`.



Project structure in Package Explorer

Open `pom.xml` and add the `spring-context` dependency from https://mvnrepository.com/artifact/org.springframework/spring-context. Always add one version below the latest.

```
<dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-context</artifactId>
    <version>6.2.19</version>
    <scope>compile</scope>
</dependency>
```

Step 3: Create the POJO classes

Create the POJO classes `Department`, `Address` and `Employee` in package `com.amazon.employee`.

Department.java

```
package com.amazon.employee;

public class Department {

    private int departmentId;
    private String departmentName;


    public Department(int departmentId, String departmentName) {
        super();
        this.departmentId = departmentId;
        this.departmentName = departmentName;
    }

    @Override
    public String toString() {
        return departmentName;
    }
}
```

Address.java

```
package com.amazon.employee;

public class Address {

    private String city;
    private String state;
    private String country;

    public Address(String city, String state, String country) {
        super();
        this.city = city;
        this.state = state;
        this.country = country;
    }

    @Override
    public String toString() {
        return "\n\t\tCity:" + city
                + "\n\t\tState:" + state
                + "\n\t\tCountry:" + country;
    }
}
```

Employee.java

```
package com.amazon.employee;

public class Employee {

    private String name;
    private String id;
    private String band;

    private Department department;
    private Address address;

    public Employee(String name, String id, String band, Department department, Address address) {
        super();
        this.name = name;
        this.id = id;
        this.band = band;
        this.department = department;
        this.address = address;
    }

    public void displayInfo() {
        System.out.println(
            "Employee Details: "
            + "\n\tName: " + name
            + "\n\tID: " + id
            + "\n\tBand: " + band
            + "\n\tDepartment: " + department
            + "\n\tAddress: " + address
        );
    }
}
```

Step 4: Create the Spring configuration class

Create the class `SpringConfig` in package `com.amazon.config`. Annotate it with `@Configuration`, and create the beans using `@Bean` methods. Inject the `Department` and `Address` beans into the `Employee` beans using `@Qualifier`.

SpringConfig.java

```
package com.amazon.config;

import org.springframework.beans.factory.annotation.Qualifier;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

import com.amazon.employee.Address;
import com.amazon.employee.Department;
import com.amazon.employee.Employee;

@Configuration
public class SpringConfig {

    @Bean("developerDepartment")
    public Department createDepartment1() {
        Department department = new Department(155, "Developer");
        return department;
    }

    @Bean("bengaluruAddress")
    public Address createAddress1() {
        Address address = new Address("Bengaluru", "Karnataka", "India");
        return address;
```

```
    }


    @Bean("emp1")
    public Employee createEmployee1(
        @Qualifier("developerDepartment") Department department,
        @Qualifier("bengaluruAddress") Address address
    ) {
        Employee employee = new Employee("Alpha", "EMP-101", "PREMIUM", department, address);
        return employee;
    }


    @Bean("qaDepartment")
    public Department createDepartment2() {
        Department department = new Department(165, "QA");
        return department;
    }

    @Bean("hyderabadAddress")
    public Address createAddress2() {
        Address address = new Address("Hyderabad", "Telangana", "India");
        return address;
    }



    @Bean("emp2")
    public Employee createEmployee2(
        @Qualifier("qaDepartment") Department department,
        @Qualifier("hyderabadAddress") Address address
    ) {
        Employee employee = new Employee("Beta", "EMP-102", "REGULAR", department, address);
        return employee;
    }
}
```

Step 5: Load the beans in App.java

In `App.java` (package `com.amazon.user`), create the Spring container using `AnnotationConfigApplicationContext` and pass `SpringConfig.class` to it. Then get the beans `emp1` and `emp2` from the container and call `displayInfo()` on each.

App.java

```
package com.amazon.user;

import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;

import com.amazon.config.SpringConfig;
import com.amazon.employee.Employee;

public class App
{
    public static void main( String[] args )
    {
        System.out.println( "Creating Spring Beans using Java..." );

        System.out.println("==========================================");

        ApplicationContext context = new AnnotationConfigApplicationContext(SpringConfig.class);

        Employee employee1 = context.getBean("emp1", Employee.class);
        employee1.displayInfo();

        System.out.println("==========================================");

        Employee employee2 = context.getBean("emp2", Employee.class);
        employee2.displayInfo();
    }
}
```

Step 6: Run the application and check the output

Run `App.java` as a Java Application. The console prints both Employee beans created by Spring.

Output

```
Creating Spring Beans using Java...
==========================================
Employee Details:
    Name: Alpha
    ID: EMP-101
    Band: PREMIUM
    Department: Developer
    Address:
        City:Bengaluru
        State:Karnataka
        Country:India
==========================================
Employee Details:
    Name: Beta
    ID: EMP-102
    Band: REGULAR
    Department: QA
    Address:
        City:Hyderabad
        State:Telangana
        Country:India
```